

Pearls AQI Predictor: An End-to-End Machine Learning Pipeline for Air Quality Forecasting

Zuhanoor¹

¹Data Science Internship, 10Pearls

²Bahria University, Islamabad

August 2026

Abstract

This report documents the design and implementation of the Pearls AQI Predictor, a serverless machine learning pipeline that forecasts Islamabad’s Air Quality Index (AQI) three days in advance. The system integrates automated data collection, feature engineering, a cloud feature store, comparison of four model classes ranging from linear regression to a deep neural network, SHAP-based explainability, CI/CD automation, and a live interactive dashboard. Ridge Regression achieved the strongest next-day performance ($R^2 = 0.511$), outperforming more complex models – a result attributed to the limited size of the training dataset. We discuss the full pipeline, the engineering issues encountered along the way (including a stale upstream data source), and the implications of our results for short-horizon air quality forecasting in data-constrained settings.

1 Introduction

Air pollution in Islamabad follows a pronounced seasonal cycle, with concentrations rising sharply in the winter months due to temperature inversions, reduced wind dispersal, and regional crop-burning activity. Reliable short-term forecasting of the Air Quality Index (AQI) can support public health decisions, yet most publicly available forecasts are limited to single-point current readings rather than forward-looking predictions.

This project was undertaken as part of a data science internship at 10Pearls, with the objective of building a complete, serverless machine learning system capable of forecasting AQI one to three days ahead for Islamabad. The system was required to span the full pipeline: automated data ingestion, feature engineering, a managed feature store, model training and evaluation, automated retraining, model explainability, and a public-facing dashboard.

2 System Architecture

The system is organised as five independent, loosely-coupled stages, each implemented as a standalone Python script and orchestrated via scheduled GitHub Actions workflows:

1. A **feature pipeline** that ingests live weather and pollutant data.
2. A **Hopsworks feature store** that persists engineered features.

3. A **training pipeline** that fits and evaluates candidate models.
4. A **model registry** that versions trained artefacts.
5. A **Streamlit dashboard** that serves live forecasts to end users.

3 Data Collection and Feature Engineering

3.1 Data Sources

Historical daily pollutant statistics (minimum, maximum, median, quartiles, and standard deviation) were obtained from the AQICN historical data platform for Islamabad, covering January 2025 to July 2026. Corresponding historical weather variables (temperature, precipitation, wind speed, and surface pressure) were retrieved from the Open-Meteo Historical Weather API for the same coordinates and date range, and the two sources were merged on date to yield 545 matched daily observations.

For live inference, the pipeline was originally built against the AQICN real-time API (Islamabad US Embassy station). During development it was discovered that this station’s live feed had not updated in several months; this was confirmed against three independent third-party AQI sources. The live data source was subsequently migrated to the Open-Meteo Air Quality and Weather APIs, which update reliably on an hourly basis and require no authentication token.

3.2 Engineered Features

From the merged dataset the following feature groups were derived:

- **Calendar features** – day of week, day of month, and month.
- **Derived feature** – a first-difference AQI change rate between consecutive days.
- **Lag features** – AQI at one, two, and three days prior.
- **Rolling features** – trailing 3-day and 7-day rolling means, computed strictly from prior observations to avoid look-ahead leakage.

Three forecast targets were constructed via forward time-shifting of the median AQI column, corresponding to one-, two-, and three-day-ahead horizons.

4 Feature Store and Model Registry

Engineered features are persisted to a Hopworks Feature Store as a versioned feature group (`aqi.features`), keyed on `date`, which also serves as the event-time column. A second version of the feature group was created after lag and rolling features were introduced, allowing the feature schema to evolve without discarding prior history. Both the training pipeline and the live dashboard read from this shared feature store, guaranteeing that the feature computation logic used at training time is identical to that used at inference time.

Trained models are registered in the Hopworks Model Registry together with their RMSE, MAE, and R^2 metrics. At inference time, the dashboard retrieves the best-performing registered model for each forecast horizon by RMSE, allowing model updates to be deployed without any change to the dashboard code.

5 Model Training and Evaluation

Four model classes were trained and compared for each of the three forecast horizons: Ridge Regression and Random Forest (scikit-learn), Gradient Boosting (scikit-learn), and a feed-forward neural network (TensorFlow/Keras) with two hidden layers of 32 and 16 units and ReLU activations. All models were trained on a chronological 80/20 split (rather than a random split) to respect the temporal structure of the data and avoid information leakage from future observations into the training set.

Table 1: R^2 by model and forecast horizon.

Model	Day 1	Day 2	Day 3
Ridge Regression	0.511	0.003	-0.157
Random Forest ¹	0.285	–	–
Gradient Boosting	0.294	0.024	-0.075
Neural Network	-0.221	-0.299	-0.412

Ridge Regression achieved the strongest performance at the one-day horizon ($R^2 = 0.511$), and was matched or exceeded only marginally by Gradient Boosting at the two- and three-day horizons, where all models perform poorly in absolute terms. The neural network underperformed all classical baselines at every horizon. We attribute this to the limited size of the training set (approximately 430 samples after feature engineering), which is insufficient for a higher-capacity model to generalise; this is consistent with the well-established requirement that deep learning models typically require substantially larger datasets than classical linear or ensemble methods to avoid overfitting.

6 Model Explainability

SHAP (SHapley Additive exPlanations) values were computed for the day-1 Ridge Regression model to attribute each prediction to its constituent features. Figure 1 shows the SHAP contribution breakdown for a representative live prediction. The most influential features were the 25th-percentile pollutant reading (**q1**), the standard deviation of same-day readings (**stdev**), and the day’s minimum reading, with same-day statistical features consistently outweighing the engineered lag and rolling-average features. This suggests that next-day AQI in Islamabad is driven primarily by the current day’s pollution distribution rather than the recent multi-day trend.

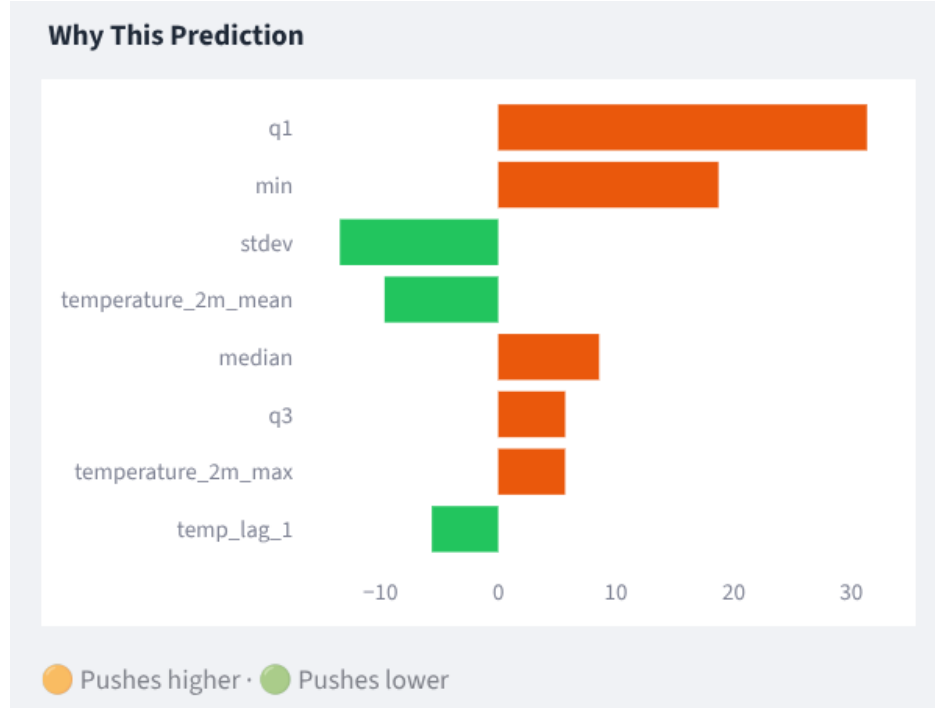


Figure 1: SHAP feature contribution breakdown for a live one-day-ahead prediction.

7 Exploratory Data Analysis

The 535-day historical series (Figure 2) exhibits a pronounced seasonal pattern, with median AQI remaining in the moderate range (approximately 20–70) through spring and summer, rising sharply from October through January, and peaking above 240 in December. This is consistent with Islamabad’s documented winter smog season. Table 2 summarises the descriptive statistics of the median daily AQI series.

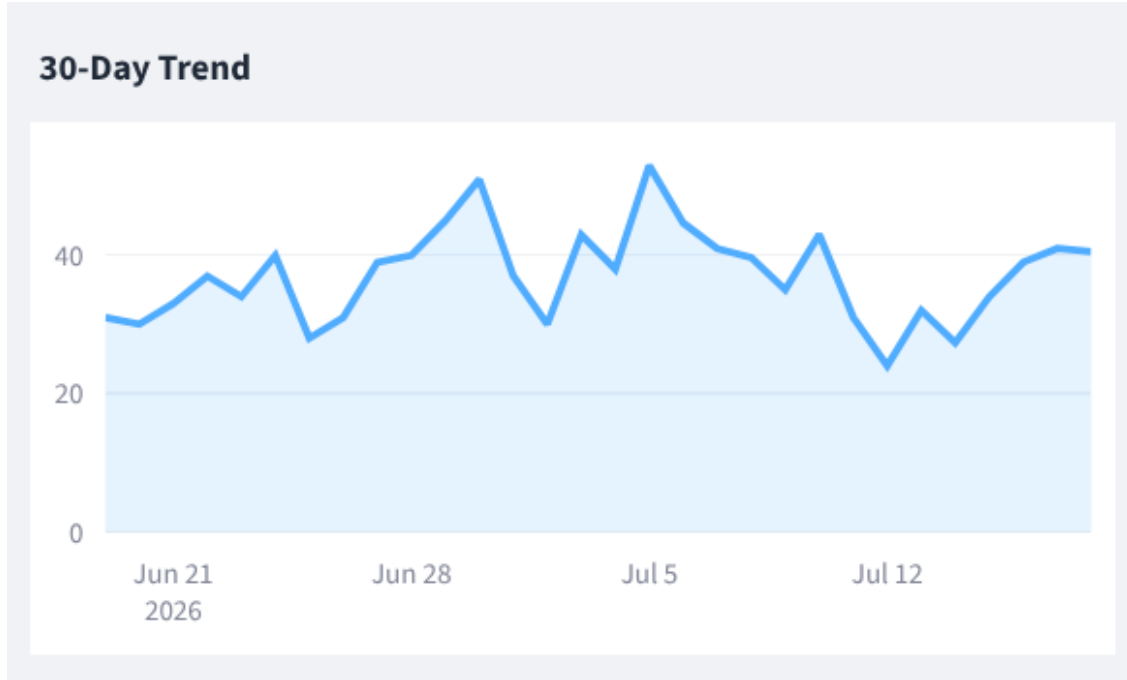


Figure 2: Daily median AQI for Islamabad, January 2025 – July 2026.

Table 2: Descriptive statistics of daily median AQI, Jan 2025–Jul 2026.

Statistic	Value
Mean	60.13
Median	46.0
Std. deviation	40.77
Minimum	4.0
25th percentile	35.0
75th percentile	70.0
Maximum	241.67

8 Pipeline Automation

The feature and training pipelines are automated using GitHub Actions, requiring no dedicated server infrastructure. The feature pipeline executes hourly, fetching live readings and committing updated data to the repository. The training pipeline executes daily, retraining all candidate models on the latest feature-store data and updating the model registry accordingly.

Hourly Feature Pipeline

feature_pipeline.yml

Filter workflow runs

...

534 workflow runs

Event ▾Status ▾Branch ▾Actor ▾

This workflow has a workflow_dispatch event trigger.

Run workflow ▾

✓

Hourly Feature Pipeline

main

Today at 1:57 PM

26s

...

Hourly Feature Pipeline #534: Scheduled

✓

Hourly Feature Pipeline

main

Today at 4:33 AM

20s

...

Hourly Feature Pipeline #533: Scheduled

✓

Hourly Feature Pipeline

main

Aug 27, 6:43 PM GMT+5

18s

...

Hourly Feature Pipeline #532: Scheduled

✓

Hourly Feature Pipeline

main

Aug 27, 7:52 AM GMT+5

19s

...

Hourly Feature Pipeline #531: Scheduled

✓

Hourly Feature Pipeline

main

Aug 27, 2:40 AM GMT+5

24s

...

Hourly Feature Pipeline #530: Scheduled

✓

Hourly Feature Pipeline

main

Aug 26, 11:56 PM GMT+5

22s

...

Hourly Feature Pipeline #529: Scheduled

Figure 3: GitHub Actions execution history for the hourly feature pipeline.

9 Web Dashboard

A Streamlit dashboard (Figures 4–5) presents the current AQI reading, a three-day forecast with categorical air-quality labels, a rolling historical trend, and the live SHAP explanation described in Section 6. The dashboard incorporates a staleness check on the live data feed: if the most recent automated reading is more than three days old, the interface falls back to the latest verified feature-store observation rather than displaying an outdated value as current. An alert banner is shown whenever the maximum three-day predicted AQI exceeds 100 (moderate concern) or 150 (hazard alert). The dashboard is deployed and publicly accessible via Streamlit Community Cloud.



Figure 4: Dashboard home view showing current AQI, category, and live weather conditions.

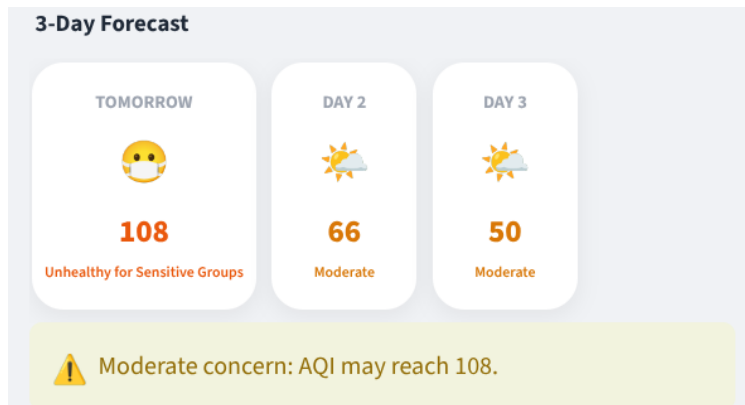


Figure 5: Three-day forecast panel with categorical AQI labels and hazard-alert banner.

10 Discussion

Two findings stand out. First, the simplest model tested, Ridge Regression, consistently outperformed ensemble and deep learning alternatives, which we interpret as a direct consequence of the small training set rather than any fundamental limitation of the more complex model classes – given a larger corpus of historical data, this ranking would likely shift. Second, forecast quality degrades sharply beyond a one-day horizon across all models, indicating that same-day and next-day pollutant statistics carry most of the predictive signal available in this dataset, and that additional exogenous features (regional pollution, traffic, or crop-burning indicators) would likely be required to improve multi-day forecasts.

11 Engineering Challenges

- **Feature-store connectivity:** initial Hopsworks connections failed due to missing optional dependencies (`pyarrow`, `confluent-kafka`) and an unsupported default storage format; resolved by installing the required extras and explicitly specifying the Hudi time-travel format.
- **CI/CD permissions:** automated commits from GitHub Actions initially failed due to insufficient token scope and repository write permissions, and required an explicit override of a `.gitignore` rule to track the live readings file.
- **Upstream data staleness:** the AQICN Islamabad station was found to have silently stopped updating; this was diagnosed by cross-referencing independent AQI sources and resolved by migrating the live feed to the Open-Meteo API.
- **Deployment dependency conflicts:** deploying to Streamlit Community Cloud surfaced Python-version incompatibilities between older `hopsworks` package builds and Python 3.14, resolved by pinning package versions and the runtime Python version.
- **Model capacity versus data volume:** the neural network and, to a lesser extent, the ensemble methods, showed clear signs of overfitting on the available training set, motivating the use of Ridge Regression as the production model for the one-day horizon.

12 Limitations and Future Work

- Two- and three-day forecasts currently carry limited predictive value and would benefit from additional exogenous features.
- The historical dataset provides only aggregate daily pollutant statistics rather than per-pollutant breakdowns (PM2.5, PM10, O₃, NO₂, SO₂, CO).
- Only a single monitoring station is available for Islamabad in the AQICN network; incorporating satellite-derived or multi-station data would improve robustness.
- Sequence models such as LSTMs may be better suited to this forecasting task given a substantially larger training corpus.

13 Conclusion

We presented a complete, serverless machine learning pipeline for short-horizon AQI forecasting in Islamabad, covering data ingestion, feature engineering, a managed feature store, multi-model training and comparison, explainability, CI/CD automation, and a live public dashboard. All requirements specified in the original project brief were met. Our results indicate that, under the data constraints of this project, a simple linear model outperforms both ensemble and deep learning alternatives at short forecast horizons – a finding with practical implications for similarly data-constrained air quality forecasting deployments.

Code and Live Demo

- **Repository:** <https://github.com/zuha20/aqi-predictor-islamabad>

- **Live Dashboard:** <https://pearls-aqi-predictorr.streamlit.app/>